

Updates and formatted output

CSC103 Programming Fundamentals / Lecture 06

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Print a two-column receipt for Pen 25.5 and Book 120.0.
Use aligned names and two decimal places.

Print each item with a fixed-width
name and a numeric price field.

Learning objectives

- Trace prefix and postfix updates
- Distinguish print, println and printf
- Format a readable numeric report

CLO-1

Increment and decrement

Compound assignment

Output methods

Format specifiers

Increment and decrement

- `x++` and `++x` both increase `x` by one.
- Postfix produces the old value. Prefix produces the updated value.
- Standalone updates are easier to read.

Example

```
int x = 4;  
int a = x++; // a=4, x=5  
int b = ++x; // x=6, b=6
```

Compound assignment

- `x += amount` updates `x` using its current value.
- The same pattern supports `-=`, `*=`, `/=` and `%`.
- Use ordinary assignment when a conversion needs explanation.

Example

```
int stock = 20;  
stock -= 3;  
stock += 5;  
// stock is 22
```

Output methods

- `print` leaves the cursor on the current line.
- `println` ends the line. `printf` uses a format string and arguments.
- `%n` inserts a platform-appropriate line separator.

Example

```
System.out.print("Total: ");  
System.out.println(24);  
System.out.printf("%d%n", 24);
```

Format specifiers

- %d formats an integer and %s formats text.
- %.2f shows two decimal places for floating-point output.
- A width reserves a minimum field size.

Example

```
System.out.printf("%-8s %6.2f%n",  
                  "Tea", 125.5);
```

A stored value and an expression result differ

- Postfix increment first produces the old value, then updates the variable.
- Prefix increment first updates the variable, then produces its new value.
- Separating updates from larger expressions makes this ordering easier to inspect.

Example

```
int x = 4;  
int old = x++;  
// old is 4, x is 5
```

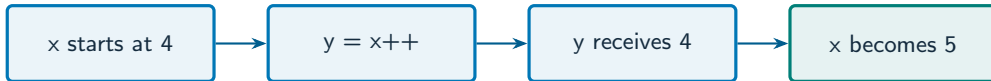

Formatted output as a small report

- A format string defines how each supplied value appears.
- Field width is a minimum width, so a longer value can still expand the field.
- A consistent decimal format lets a reader compare prices quickly.

Example

```
%-8s means left-aligned text in at least 8 positions.  
%8.2f means a number with two decimal places.
```

Postfix preserves the old expression value



What to notice

Separate the expression result from the variable state after evaluation.

Key distinctions

Format	Argument example	Displayed purpose
%d	25	Whole number
%.2f	25.5	25.50
%-8s	Pen	Left-aligned item name
%n	No argument	Platform line ending

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int count = 4;  
int before = count++;  
System.out.printf(java.util.Locale.US,  
    "%d %d %.2f%n", before, count, 12.5);
```

First example: result

4 5 12.50

before receives 4, then count becomes 5.

The floating-point value displays two decimal places.

Output methods
Format specifiers

Guided practice

Print a two-column receipt for Pen 25.5 and Book 120.0.
Use aligned names and two decimal places.

Attempt the solution before
continuing.
Use the definitions and examples
from this lecture.

Solution strategy

- Print each item with a fixed-width name and a numeric price field.
- Use `Locale.US` to keep the demonstrated decimal point consistent.
- Calculate the total separately, then format it with the same price pattern.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution06 {  
    public static void main(String[] args) throws Exception {  
        double pen = 25.5;  
        double book = 120.0;  
        System.out.printf(java.util.Locale.US, "%-8s %8.2f%n", "Pen", pen);  
        System.out.printf(java.util.Locale.US, "%-8s %8.2f%n", "Book", book);  
        System.out.printf(java.util.Locale.US, "%-8s %8.2f%n", "Total", pen + book);  
    }  
}
```


Execution trace

Item	Stored price	Displayed price
Pen	25.5	25.50
Book	120.0	120.00
Total	145.5	145.50

Follow the changing state in execution order.

Solution result

Pen 25.50
Book 120.00
Total 145.50

Why this result follows
Calculate the total separately, then
format it with the same price
pattern.

A common incorrect approach

```
System.out.printf("Price: %d", 12.5);
```

Example

%d expects an integral value. Use %.2f for this double value.

Boundary and failure checks

Check the stated input assumptions.

Create a marks report showing a name, integer marks and percentage to one decimal place.

Use %-8s for each item name and
%8.2f for each price, followed by %n.
Total: 145.50.

Check your understanding

With $x=2$, what does $y=++x$ do? Does `%.2f` round the stored variable?

Write a prediction and explain the rule that supports it.

Answer and explanation

x becomes 3 and y receives 3. Formatting rounds the displayed representation, not the stored variable.

%d expects an integral value. Use %.2f for this double value.

Compare cases and predict outcomes

Statement (fresh $x=4$)	Assigned y	Final x
$y = x++$	4	5
$y = ++x$	5	5
$x++; y = x$	5	5

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Trace $x=4$, $y=x++$, $z=++x$. Then print y , z and x in labelled columns.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int x = 4;  
int y = x++;  
int z = ++x;  
System.out.printf("y=%d z=%d x=%d\n", y, z, x);
```

- Postfix stores the old value in y, then x is 5.
- Prefix increments x to 6 and supplies 6 to z.
- Output: y=4 z=6 x=6.

Tracing four update operators

- Java evaluates these operands left to right, so each update affects later operands.
- Record the value supplied by an operand separately from the new value of `i`.
- Such expressions are useful trace exercises; separate updates are easier to maintain.

Source: [supplied ISB campus slides](#). See [speaker notes](#) and [ISB_Source_Map.md](#).

Worked problem: Tracing four update operators

Task

Trace `int i=5` and `result=++i+i--+i++++--i`, written with spaces in the program. Predict both printed values.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture06 {  
    public static void main(String[] args) throws Exception {  
        int i = 5;  
        int result = ++i + i-- + i++ + --i;  
        System.out.println("Result: " + result);  
        System.out.println("Final i: " + i);  
    }  
}
```

Worked trace and expected result

Operand	Value supplied	i afterward
++i	6	6
i--	6	5
i++	5	6
--i	5	5

Expected console output

Result: 22

Final i: 5

Extend the ISB example

Practice

Replace the expression with four named intermediate values and verify the same total.

Explain your reasoning

Store each operand in sequence, then add $6 + 6 + 5 + 5 = 22$. i ends at 5.

Lecture summary

- prefix and postfix updates
- print, println and printf
- Format a readable numeric report
- Print each item with a fixed-width name and a numeric price field.
- Use Locale.US to keep the demonstrated decimal point consistent.
- Calculate the total separately, then format it with the same price pattern.

After-class practice

Create a marks report showing a name, integer marks and percentage to one decimal place.

Syllabus reading

Liang Ch 2

Next lecture: Boolean expressions